The International Journal of Advanced Manufacturing Technology (2025) 140:5617–5636
https://doi.org/10.1007/s00170-025-16606-4

ORIGINAL ARTICLE

Knowledge-guided deep reinforcement learning framework for feature machining step sequencing

Pengyu Wang[1] · Youpeng You[2] · Weichao Liu[3] · Dawei Ding[1] · Fengyu Xu[1]

Received: 19 February 2025 / Accepted: 14 September 2025 / Published online: 26 September 2025
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2025

Abstract
Machining sequencing is a critical step in computer-aided process planning (CAPP) for numerical control (NC) machining, directly impacting efficiency and cost. Consequently, the machining sequencing method is crucial for practical manufacturing processes. However, existing sequencing methods face two main limitations: (1) they often treat machining features (MFs) as the smallest unit, neglecting that complex features require decomposition into sub-features and multiple feature machining steps (FMSs); (2) while exact methods are very time-consuming, prevailing approximate methods suffer from inconsistent performance, repeated iterative computations, and an inability to learn from experience. To address these challenges, this paper proposes a knowledge-guided deep reinforcement learning (DRL) framework for FMS-level sequencing. The method incorporates dynamic selection masks to enforce machining constraints and a process knowledge-based cost model to ensure efficient convergence. Leveraging the sequencing experience acquired during the learning process, this method can solve the machining sequencing problem within an acceptable computation time. Experiments demonstrate that the proposed method outperforms existing methods in both total cost and computation time, enabling more efficient machining of complex parts.

Keywords Machining features · Feature sequencing · Reinforcement learning · Intelligent manufacturing

1 Introduction

To reduce the time and production costs associated with product design and manufacturing, manufacturers widely adopt digital solutions based on computer-aided technologies. For instance, computer-aided design (CAD) [1] enables rapid part modeling, while computer-aided manufacturing (CAM) reduces physical prototyping costs through virtual simulation. Within this suite of solutions, decision-making in computer-aided process planning (CAPP) is particularly critical, as it directly affects the efficiency and cost of the final machining process. A CAPP system typically comprises multiple modules, such as feature recognition [2], machining resource specification [3], machining parameter determination [4], machining operation sequencing [5], and process model construction [6]. Among these modules, this paper focuses specifically on the sequencing of feature machining steps (FMSs) within machining operation sequencing, as it represents a core task in achieving low-cost and high-efficiency machining, posing significant research challenges and value.

In the field of machining operation sequencing, machining features (MFs) [7] are widely used to define part geometric regions with machining semantics. To achieve both high efficiency and high precision, a typical strategy involves first applying high-efficiency rough machining steps to a feature, followed by high-precision finishing steps [8]. However, transitions between FMSs may cause fixture adjustments, tool replacement, and non-cutting tool paths. Specifically, fixture adjustments involve repositioning parts for different tool access directions (TADs) [9]. Tool replacement is to meet the requirements of different FMSs. Non-cutting tool paths refer to the idle movements between different features [10]. These factors contribute to non-cutting time costs, thereby affecting overall machining efficiency. Therefore, to reduce non-cutting time during the machining process, it is essential to optimize the sequencing of FMSs.

✉ Fengyu Xu
13915964356@njupt.edu.cn

[1] College of Automation, Nanjing University of Posts and Telecommunications, Nanjing 210023, China

[2] College of Mechanical and Electrical Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing 210016, China

[3] School of Mechanical Engineering, Nanjing Institute of Technology, Nanjing 211167, China

In recent years, significant research has been conducted to address optimization challenges in machining operation sequencing. Research solutions can be categorized into exact and approximate methods. Exact methods, such as linear programming [11], dynamic programming [12], and branch-and-bound [13], seek the optimal sequencing result by exploring the entire solution space. However, as the complexity of the problem increases, the solution space expands exponentially, making exact methods computationally expensive for complex sequencing tasks. To improve the speed of exact methods, scholars have integrated process rules and knowledge to enhance the search process. Wang et al. [14] developed a rule and branch-and-bound algorithm to reduce non-cutting tool paths between features. Although the rules reduce the solution space, the algorithm still searches the sub-solution space for the same type of features. Consequently, the computational time remains high. Hao et al. [15] aim to minimize the deformation of parts by optimizing the sequencing of machining operations using dynamic programming. Deformation rules are extracted to speed up optimization. However, the computational time of dynamic programming is still affected by the number of features. Despite improvements, the computation time for exact sequencing remains insufficient for practical manufacturing demands.

To improve computation time, researchers also explored approximate methods based on swarm intelligence [16] and evolutionary algorithms that aim to produce near-optimal solutions within acceptable times. For example, Su et al. [17] proposed an operation sequencing based on the genetic algorithm [18] and edge selection. The potential operation sequence is represented by the genes based on the encoding strategy. The edge selection method guarantees feasible initial solutions, thereby avoiding precedence constraint violations and enhancing search efficiency. Wen et al. [19] and Neumann et al. [20] both improved genetic algorithms for process planning. Hu et al. [21] developed a constraint-aware ant colony optimization method for operation sequencing. To enhance global search and avoid local optima, the method integrates adaptive updating and local search mechanisms. Xiao et al. [22] solved the energy optimization problem in process planning based on the multi-objective imperialist competitive algorithm. Zhang et al. [23] proposed an improved spider monkey optimization algorithm that combined mixed-integer programming to address the sequence issues. Liu et al. [24] designed an improved genetic algorithm combining a neighborhood searching strategy for the integrated process planning and scheduling problem. Xu et al. [25] proposed a hybrid ant colony algorithm and simulated annealing approach to generate the global optimal roughing tool sequence. However, the above approximation methods are mainly based on swarm intelligence and evolutionary algorithms and have obvious limitations, including falling into local optima and exhibiting unstable performance.

The rapid advancement of deep learning has opened up new possibilities for addressing complex optimization problems. Zhang et al. [26] developed a process decision-making approach based on deep learning and knowledge graph. The process reasoning function is learned by the deep learning model. Han et al. [27] proposed a machining step sequence generation model based on deep learning to predict the candidate machining step and its machining regions. Wang et al. [28] solved the feature process planning problem with the graph convolutional neural networks. Zhang et al. [29] proposed a novel method based on deep reinforcement learning (DRL) for machining process route planning. Based on the above work, Zhang et al. [30] developed an improved DRL-based framework for machining process planning, specifically targeting aircraft structural parts. Xiao et al. [31] also proposed a process planning reasoning method based on deep reinforcement learning. The aforementioned studies demonstrate that deep learning models are capable of effectively resolving various optimization problems.

Inspired by the above developments, this paper proposes a DRL framework guided by process knowledge to optimize the sequencing of FMSs. The key innovations of the framework include the following: (1) formulating the sequencing problem at the FMS level, acknowledging the limitations of MF-level approaches; (2) constructing the DRL framework which can learn sequencing experience through unsupervised learning of parts; (3) introducing a process knowledge-based cost model incorporating both local and global costs to effectively guide the DRL agent; and (4) employing dynamic selection masks within the DRL framework to guarantee compliance with all machining constraints throughout the sequencing process.

2 Cost optimization of FMS sequencing

2.1 Sequencing problem description

Figure 1 presents the process of sequencing FMSs. The process personnel specify the feature process chain composed of several FMSs for each MF. This paper assumes that the parts can be processed on a single machine tool. This paper defines a machining operation in the feature process chain as an FMS. The FMS types include roughing and finishing. If all FMSs are sequenced feature by feature, frequent fixture adjustments, cutting tool replacements, and non-cutting tool movements will occur. To reduce sequencing time and improve machining efficiency, an appropriate sequencing method is necessary.

The problem of FMSs sequencing can be mathematically described as follows:

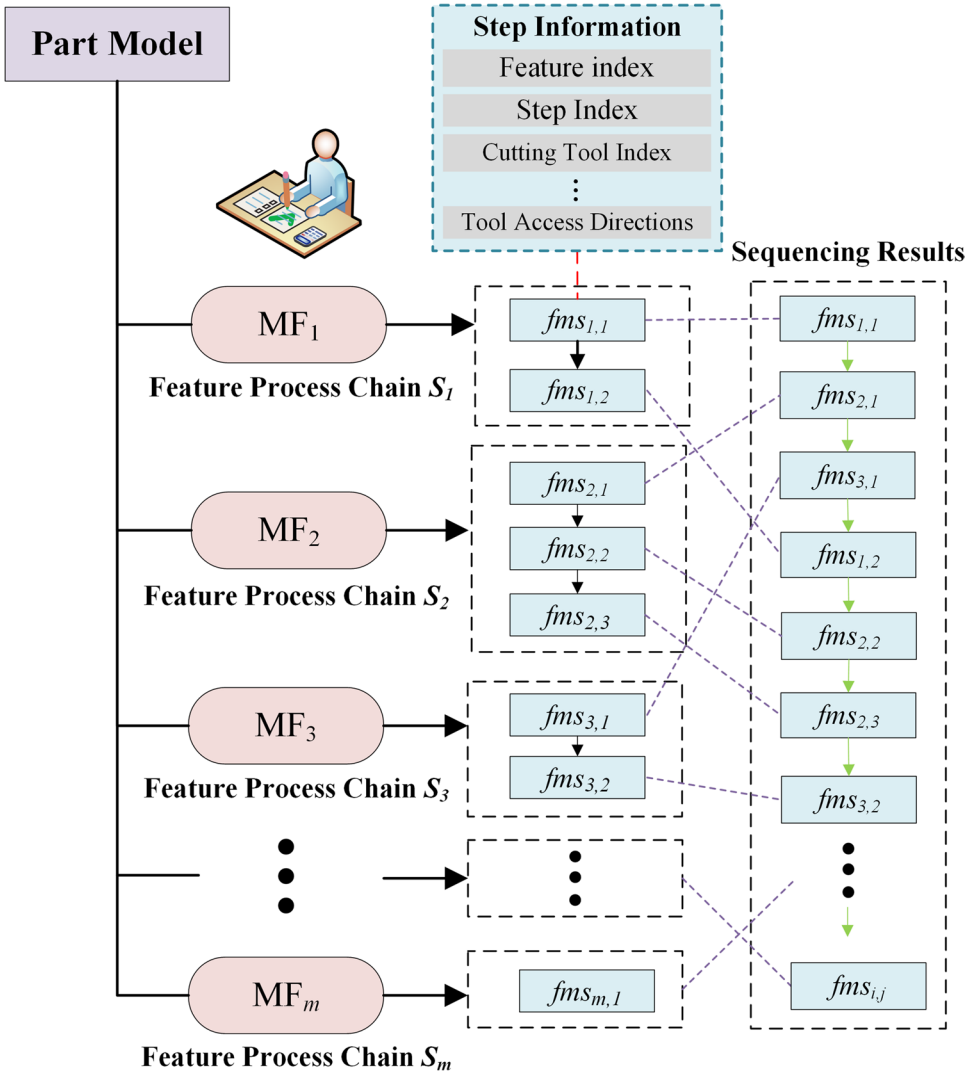


Fig. 1 Manual sequencing of feature machining steps

- For a part, the set of all machining features is denoted as F, expressed as:

$$F = \{f_i\}_{i=1,2,\ldots,m} \tag{1}$$

where f_i is the ith machining feature, and m is the total number of machining features in the part.

- For each machining feature f_i, all of its FMSs form a feature process chain S_i in sequence, expressed as:

$$S_i = \{fms_{ij}\}_{j=1,2,\ldots,n_i} \tag{2}$$

where fms_{ij} is the jth FMS of the feature process chain of the ith feature, n_i is the total number of FMSs of the feature process chain of the ith feature.

- The TADs of all machining features form a set A, expressed as:

$$A = \{a_k\}_{k=1,2,\ldots,p} \tag{3}$$

where a_k is the kth TAD, and p is the total number of TADs in the set.

- The cutting tools of all FMSs form a set T, expressed as:

$$T = \{t_h\}_{h=1,2,\ldots,q} \tag{4}$$

where t_h is the hth cutting tool, and q is the total number of cutting tools in the set.

2.2 Cost model based on process knowledge

According to process knowledge, the switching of FMS in sequencing results leads to non-cutting time costs. The total non-cutting time cost includes the time spent on fixture adjustment, cutting tool replacement, and

non-cutting tool paths. Based on the process knowledge, the total non-cutting time cost C can be specifically modeled as:

$$C = c_1 + c_2 + c_3 \tag{5}$$

where c_1 is the cost of fixture adjustment, c_2 is the cost of cutting tool replacement, and c_3 is the cost of non-cutting tool paths, which are specifically expressed as:

$$c_1 = \lambda_1 \sum_{i=1}^{n} \sum_{j=1}^{n} e_{ij} \left(1 - \sum_{k=1}^{p} a_i^k a_j^k \right), \ i \neq j \tag{6}$$

$$c_2 = \lambda_2 \sum_{i=1}^{n} \sum_{j=1}^{n} e_{ij} \left(1 - \sum_{h=1}^{q} t_i^h t_j^h \right), \ i \neq j \tag{7}$$

$$c_3 = \lambda_3 \sum_{i=1}^{n} \sum_{j=1}^{n} e_{ij} \sum_{k=1}^{p} a_i^k a_j^k \sum_{h=1}^{q} t_i^h t_j^h dis(i,j), \ i \neq j \tag{8}$$

where λ_1 is the cost coefficient of fixture adjustment, λ_2 is the cost coefficient of cutting tool replacement,λ_3 is the cost coefficient of non-cutting tool paths, n is the total number of all FMSs, and e_{ij} represents the FMS sequence variable; when jth FMS follows ith FMS (i.e., $j = i + 1$), the value of e_{ij} is 1; otherwise, the value of e_{ij} is 0; a_i^k is the TAD variable; when the direction k is same as the TAD of ith FMS, the value of a_i^k is 1; otherwise, the value of a_i^k is 0. The definition of a_j^k can be derived similarly; t_i^h is the cutting tool variable; when ith FMS uses the cutting tool h, the value of t_i^h is 1; otherwise, the value of t_i^h is 0. The definition of t_j^h can be derived similarly; $dis(i,j)$ is the distance function measuring the non-cutting tool path distance between the MFs corresponding to ith and jth FMS. It is important to note that this paper does not consider the time cost associated with cutting tool changes due to tool wear. Firstly, predicting tool wear is a complex and challenging issue, and discussing it within the scope of this paper would not be appropriate. Secondly, the focus of this paper is on minimizing non-cutting toolpath, as addressed in reference [14], while also considering fixture and cutting tool change times.

The optimization aims to minimize fixture adjustments, cutting tool replacements, and non-cutting tool paths in FMS sequencing, thereby reducing overall cost. Conflicts between optimization objectives arise, as optimizing one may compromise another. For example, a "depth-first" strategy may reduce non-cutting tool paths but increase tool replacements. Typically, compromises are made by assigning weight coefficients. This paper also introduces coefficients λ_1,λ_2, and λ_3 to convert multi-objective problems into a single-objective one. This strategy converts various types of costs into dimensionless total time costs. Statistical analysis of numerous machining examples provides process knowledge and data to determine these coefficients. According to the reference [32], the coefficient values are as follows: $\lambda_1 = 100$, $\lambda_2 = 20$, $\lambda_3 = 0.05$.

The function $dis(i,j)$ of the non-cutting tool path is detailed below. Assume that the machining object of ith FMS is feature u, and the machining object of jth FMS is feature v.

(1) When feature u and feature v are identical:

$$dis(i,j) = 0 \tag{9}$$

(2) When feature u is the preceding geometric feature (PGF) of feature v:

$$dis(i,j) = \sqrt{(x_u - x_v)^2 + (y_u - y_v)^2 + (z_u - z_v)^2} \tag{10}$$

where (x_u, y_u, z_u) is the center point of the top surface of the oriented bounding box (OBB) of feature u, and (x_v, y_v, z_v) is the center point of the top surface of the OBB of feature u. This calculation method refers to the non-cutting tool path calculation method in reference [14] to solve feature sequencing. PGF indicates that feature u is the upper adjacent feature of feature v, requiring the upper feature to be processed before the lower feature. For such features with geometric dependencies, when switching the machining objects of the FMSs, it moves directly along the straight line defined by the spatial center points, reducing the non-cutting path by eliminating unnecessary tool lifts.

(3) When features u and v do not meet the above two conditions:

$$dis(i,j) = \sqrt{(x_u - x_v)^2 + (y_u - y_v)^2} + |z_u - z_v| + 2Z_{lift} \tag{11}$$

where Z_{lift} is the safe tool lifting distance. Process knowledge and machining experience dictate that to prevent collisions between the tool and the part during work step transitions, the cutting tool will be lifted a sufficient distance Z_{lift} in the vertical direction before moving horizontally. It should be noted that the part may need to be fed from TADs, not solely from the $+Z$ direction. However, this does not affect the calculation of the above function. Because all machining features were rotated to the $+Z$ direction for recognition. The relative pose between features in the same feed direction remains unchanged.

2.3 Constraints determined by process knowledge

In the actual machining process, the FMS sequencing results are subject to some machining constraints and cannot be sequenced arbitrarily. The constraints are determined by the rules of process knowledge. Process knowledge indicates that the constraints include preceding geometric feature constraints, preceding process feature constraints, and process chain constraints. Assume that the machining object of the *i*th FMS is feature *u*, the step-index in the sequencing result is ID_{i}, the machining object of the *j*th FMS is feature *v*, and the step-index in the sequencing result is ID_{j}. The knowledge-determined constraints that the FMS sequencing results must meet are summarized as follows:

(1) Preceding geometric feature constraints

If feature *u* is the PGF of feature *v*, then FMS *i* must precede FMS *j*. PGF constraint implies a geometric dependency between features, where feature *u* must be processed before feature *v*. For example, interacting features are decomposed into several sub-features layer by layer. Then the upper sub-feature is the PGF of the lower sub-feature in the same TAD. $pre_{\mathrm{uv}} = 1$ indicates that feature *u* is a PGF of feature *v*, and the constraint can be expressed as:

$$ID_i < ID_j, if\ pre_{uv} = 1 \tag{12}$$

(2) Preceding process feature constraints

If feature *u* is the preceding process feature (PPF) of feature *v*, then FMS *i* must precede FMS *j*. PPF constraint implies a process dependency between features, where feature *u* must be processed before feature *v*. For example, feature *u* is the processing reference for feature *v*. The process reference between features is specified by process personnel based on the process requirements. $base_{\mathrm{uv}} = 1$ indicates that feature *u* is a PPF of feature *v*, and the constraint can be expressed as:

$$ID_i < ID_j, if\ base_{uv} = 1 \tag{13}$$

(3) Process chain step constraints

The steps in the sequencing results must satisfy the process chain step constraints, ensuring they maintain the relative order as defined in the process chain. Assuming the index of the j_1th FMS in the process chain of feature *i* in the sequencing results is $ID_{\mathrm{ij_1}}$, and the index of the j_2th FMS of feature *i* in the process chain in the sequencing results is $ID_{\mathrm{ij_2}}$, the process chain step constraint can be expressed as:

$$ID_{ij_1} < ID_{ij_2},\ if\ j_1 < j_2 \tag{14}$$

3 Proposed sequencing framework

This methodology section is structured to provide a logical and comprehensive description from a high-level framework overview to detailed component-level implementation. The logical hierarchy is organized as follows: (1) Sect. 3.1 begins with a framework overview, illustrating the overall architecture and the relationship between the actor and critic networks; (2) Sect. 3.2 subsequently delves into the detailed component implementation of the framework, covering input representation, key network modules, the dynamic masking mechanism for constraint handling, and the optimization process; and (3) Sect. 3.3 describes the dataset construction process, which provides the necessary foundation for model training and validation. This structured approach ensures a systematic presentation of the proposed methodology, from conceptual design to practical implementation and experimental preparation.

3.1 Framework overview

Figure 2 presents the knowledge-guided DRL framework for FMS sequencing. This framework is inspired by the foundational concepts of the pointer network model [33]. The learning framework can be divided into two parts: actor network and critic network.

The actor network functions as a decision-making entity. Given the input of disordered FMS information $\{S_i\}_{i=1,2,\ldots,n}$, it outputs optimized FMS sequencing results, and the FMS information is also the input to the actor network. The actor network comprises three main components: (1) actor mapping layer, (2) recurrent encoder, and (3) recurrent decoder. The actor mapping layer utilizes a linear layer to transform S_{i} from a low-dimensional space to a high-dimensional space, resulting in a feature vector F_{i}. To address sequence conversion problems, the encoder-decoder framework is employed. This framework is made up of two recurrent neural network (RNN) modules. The RNN unit within this framework is built based on the long short-term memory (LSTM) module. In the recurrent encoder, A_1 is an LSTM module, and A_2 includes an LSTM module, an AR module, and a Pt module.

The critic network functions as a prediction network, estimating the total cost for the optimized sequencing result based on the input unordered FMSs. The first half of the critic network is similar to the actor network. It comprises three main components: (1) critic mapping layer, (2)

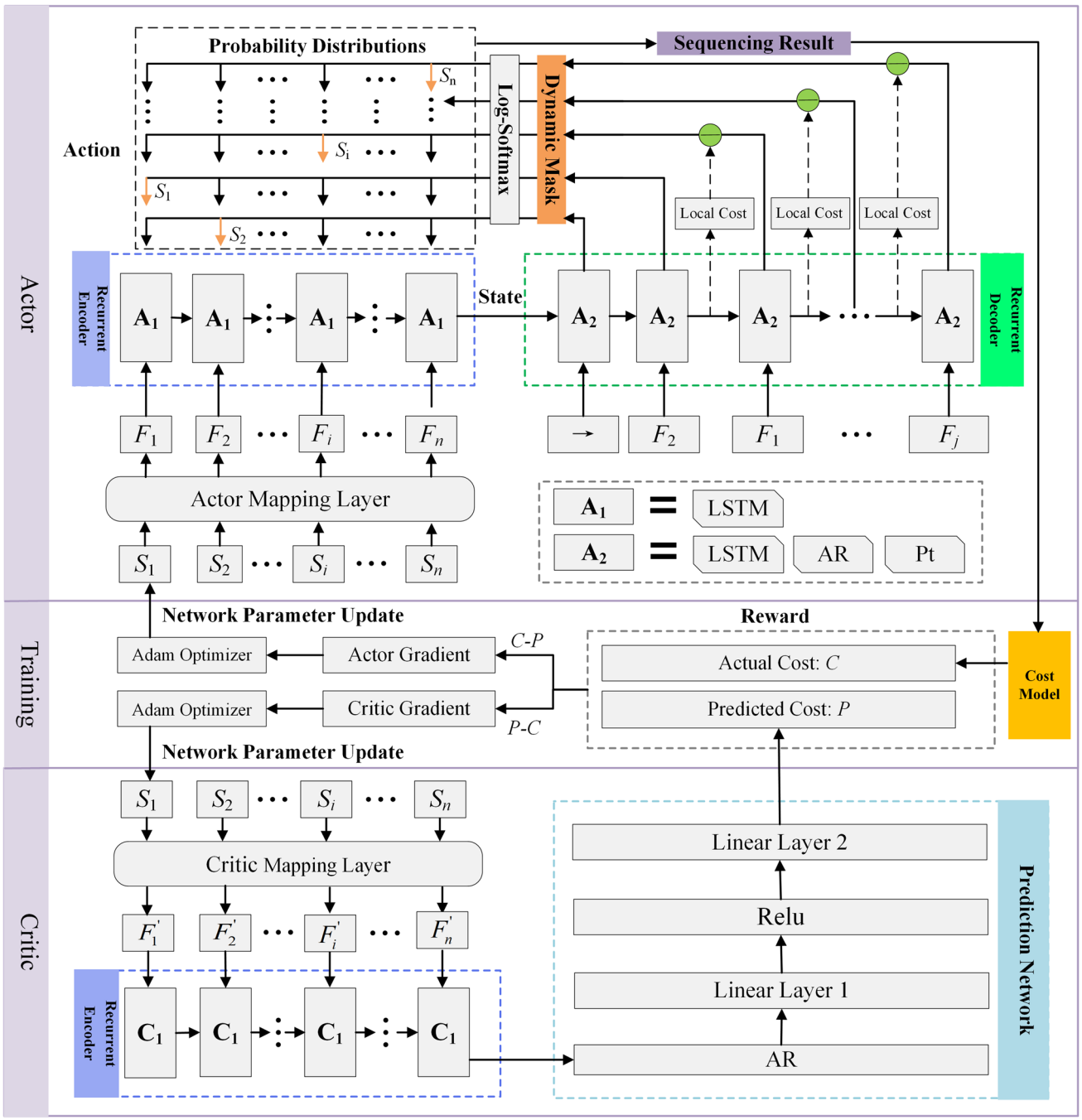


Fig. 2 Knowledge-guided DRL framework for FMS sequencing

recurrent encoder, and (3) prediction network. The structure of the critic mapping layer aligns with the mapping layer of the actor network. In the recurrent encoder, C_1 is also implemented with an LSTM module. The prediction network includes an AR module, a ReLU activation function, and two linear layers.

Reinforcement learning training of actor and critic networks for FMS sequencing is guided by process knowledge. Knowledge guidance is mainly reflected in three aspects: (1) input representation of FMS, (2) cost model, and (3) machining constraints. Leveraging comprehensive process knowledge, an input vector format that fully encapsulates the geometry and process information is designed. This vector format is detailed in Sect. 3.2.1. The coefficients in the cost model are also determined based on process knowledge and statistics. Both local cost and global cost are employed to guide the training process. The local cost helps accelerate the convergence speed of the training process, whereas the global cost aids in avoiding local

Table 1 Input vector format

Dimension	Variable	Interpretation
1	X	X coordinate value
2	Y	Y coordinate value
3	Z	Z coordinate value
4	Feature_Step_Index	Step index in all FMSs
5	Feature_Index	Index of the current feature
6	Parent_Feature_Index	Index of the parent feature
7	Pre_Geom_Feature_Index	Index of the preceding geometric feature
8	Pre_Proc_Feature_Index	Index of the preceding process feature
9	Chain_Step_Index	Step index in the feature process chain
10	TAD_Index	Index of TAD
11	Tool_index	Index of the cutting tool

optima. Furthermore, process knowledge also determines the machining constraints that the sequencing results must satisfy. A dynamic mask module is integrated into the learning framework to ensure that the next FMS selected by the actor network adheres to all constraints. The detailed construction of the dynamic mask is presented in Sect. 3.2.4.

During the training process, the objective of the actor network is to minimize the actual cost C of the sequencing result, while the critic network aims to accurately estimate the predicted cost P of the current actor network. In the initial training phase, the actor network does not produce an optimized sequencing result; however, the actual cost C is accurate, whereas the error of P is significant. The mean square error, calculated from the P–C difference, trains the prediction network and reduces the prediction error. The predicted cost P is used as the baseline. Once the prediction error is reduced, the C–P error becomes more accurate. This reduces the likelihood that the actual cost C of the sequencing result exceeds the predicted cost P, which is considered optimal. Consequently, it increases the likelihood that the actual cost C is less than the predicted cost P. The actor and critic networks engage in a dynamic mutual learning process. With an increasing number of reinforcement learning iterations, the Adam optimizer refines the parameters of both networks, leading their outputs to converge toward a relatively optimal cost level.

The specific parameters of the modeling are as follows: (1) The number of neural units in the mapping layer of the actor and critic network is 128; (2) the input dimension and the hidden state vector dimension of all LSTM units in the model are 128; (3) the number of neurons in the four attention layers within the AR module is 128; (4) the dimensions for the input and output vectors of linear layer 1 are set to 128; for linear layer 2, the input dimension is 128, while the output dimension is 1.

Building upon the above architecture, the proposed framework introduces several key innovations that significantly advance the state-of-the-art in machining sequencing. First, it constructs a novel DRL framework specifically designed for FMS-level sequencing, moving beyond traditional MF-level approaches. Second, it integrates machining process knowledge through a uniquely designed reward function that combines local and global cost models to guide the DRL agent more effectively. Third, it implements a dynamic action masking mechanism based on real-time machining constraints to ensure sequence feasibility and enhance robustness. Finally, the framework learns generalizable sequencing policies, enabling efficient experience reuse and eliminating the need for repetitive iterative computations required by traditional methods. These innovations collectively enhance the performance, efficiency, and practicality of the proposed approach for complex manufacturing scenarios.

3.2 Framework implementation

3.2.1 Input vector representation

The input S_i for FMS to both the actor and critic networks is a fixed-size vector. Informed by process knowledge, the vector format shown in Table 1 is formulated. The first three dimensions represent the coordinates (X, Y, Z) of the top center of the OBB of the feature that has been rotated to align with the $+Z$ direction. The fourth dimension denotes the FMS index, the fifth dimension indicates the feature index of the associated FMS, and the sixth dimension specifies the parent feature index. If the feature is a sub-feature derived from a parent feature, the sixth dimension is an integer greater than or equal to 0. If the feature is independent, the index value is -1. The seventh and eighth dimensions indicate the indices of preceding geometric and process features for the current FMS, respectively; if no preceding feature exists, the index is -1. The ninth dimension denotes the index of the FMS within its feature process chain, assigned according to its machining sequence. The tenth dimension indicates the TAD index of the FMS. The eleventh dimension represents the cutting tool index.

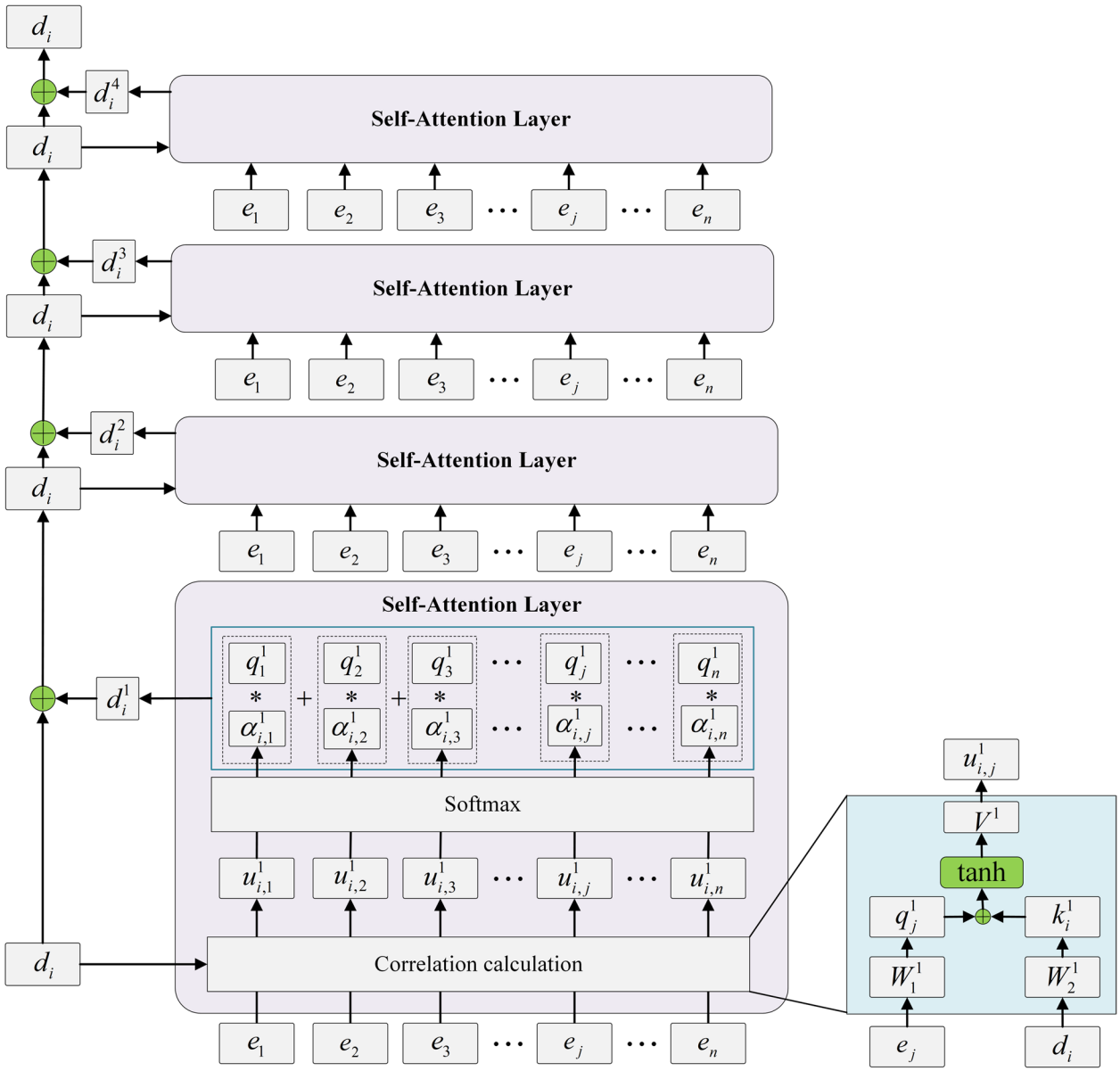


Fig. 3 Neural network structure of AR module

3.2.2 AR network module

In addition to LSTM, A_2 also includes an AR neural network module and a Pt neural network module. The AR module is a multi-layered self-attention network integrated with the residual structure, as illustrated in Fig. 3.

e_j represents the hidden state vector of all outputs of A_1, and d_i is the hidden state vector by the LSTM module of A_2 during the ith recurrent calculation. The parameters of W_1^l, W_2^l, and V^l in the AR module are all learnable, and l is the number of layers of the self-attention network. Guided by process knowledge, a four-layer self-attention network corresponding to the fixture, cutting tool replacement, non-cutting paths, and constraints has been designed to adaptively determine the relevance between the FMSs. The computation process for the lth layer of the self-attention network is detailed as follows:

$$q_j^l = W_1^l e_j \tag{15}$$

$$k_i^l = W_2^l d_i \tag{16}$$

$$u_{i,j}^l = V^l \tanh(q_j^l + k_i^l) \tag{17}$$

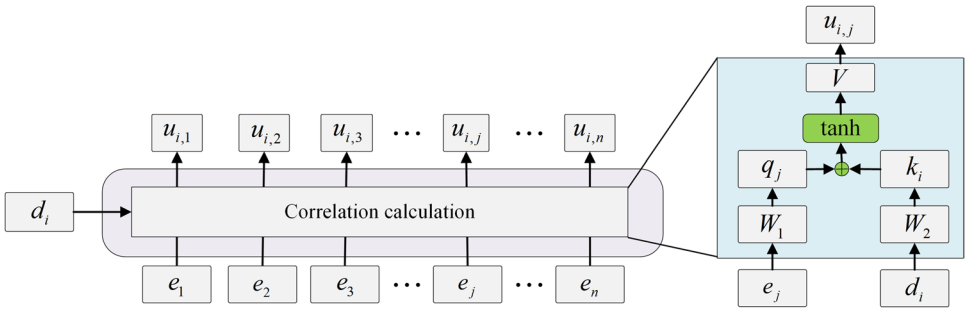


Fig. 4 Neural network structure of Pt module

$$\alpha_{i,j}^{l} = \mathrm{softmax}(u_{i,j}^{l}) = \frac{u_{i,j}^{l}}{\sum_{k=1}^{n} u_{i,k}^{l}} \tag{18}$$

$$d_i^{l} = \sum_{j=1}^{n} \alpha_{i,j}^{l} q_j^{l} \tag{19}$$

The residual network structure enables the results of the self-attention layer to be superimposed on d_i for iterative updates, and the updated results are input to the next self-attention layer. It allows the associated knowledge extracted from each self-attention layer to be continuously preserved in d_i^l. The iterative update of decoding input d_i can be represented as:

$$d_i = d_i + d_i^{l} \tag{20}$$

3.2.3 Pt network module

As illustrated in Fig. 4, the Pt neural network module is analogous to the AR neural network module. The Pt module evaluates each FMS based on the selected FMSs.

e_j represents the hidden state vector of all outputs of A_1, and d_i is the updated output of the AR module corresponding to ith step selection. The correlation between d_i and e_j is calculated to derive the evaluation value $u_{i,j}$. An FMS with a higher value $u_{i,j}$ is more likely to be selected as the next step. As shown in Fig. 2, the subsequent calculation process incorporates local cost and a dynamic mask to refine each step's evaluation value. The local cost $f_{i,j}$ is determined using the cost model to evaluate the switching cost from FMS i to FMS i. The dynamic mask m_j assigns a maximum value to the FMS that cannot be selected as the next step and assigns 0 to the remaining steps. The refined evaluation value $u_{i,j}'$ is then utilized to calculate the probability distributions of the FMSs to be selected. The specific calculation process can be expressed as follows:

$$u'_{i,j} = u_{i,j} - f_{i,j} - m_j \tag{21}$$

$$p_{i,j} = \text{Log - Softmax}(u'_{i,j}) = log(\frac{e^{u'_{i,j}}}{\sum_{k=0}^{n} e^{u'_{i,k}}}) \tag{22}$$

From the aforementioned formula for calculating the probability, it is evident that when the mask value m_j for FMS j is a maximum value, the value of $P_{i,j}$ approaches 0, ensuring that the output will not select FMS j. When the mask value m_j for FMS j is set to 0, it preserves the original probability of selecting that step. The specific calculation method of the dynamic mask is described in Sect. 3.2.4. Local cost $f_{i,j}$ introduces a local optimization operator, enhancing the probability of choosing steps with lower switching costs, thereby accelerating convergence during network training. Since the final objective function computes the costs for all steps, the local cost will not lead to the local optimum. The local cost calculation draws from the cost model in Sect. 2.2, treating it as a two-step FMS sequencing problem.

3.2.4 Dynamic mask

The Pt module employs dynamic masks in its subsequent calculations. Dynamic masks indicate the steps that cannot be selected in the current action, including those that have already been selected and those that do not satisfy constraints. The construction of the dynamic mask M_i for the ith step decision is illustrated in Fig. 5.

Initially, the total number n of all steps is determined according to FMSs. An empty set M_i is created to store the mask values corresponding to each FMS. Then traverse each FMS to calculate the mask value m_j for the jth FMS. If the jth FMS is in the processed step set MS_i, m_j is assigned a maximum value. Otherwise, further determination of the mask value m_j is required. Based on the information of the jth FMS, its corresponding feature index a is extracted. If all the steps of the PGF of feature a are not in MS_i, it signifies that the geometric constraints of the jth FMS are unmet, and the mask value m_j receives a maximum value. If all the steps of the PPF of feature a are not in MS_i, m_j is also assigned a maximum value. Otherwise, evaluate whether the process chain step constraint is satisfied. The step index b corresponding to the jth step in its process chain is obtained

based on the step information. If steps before b in the process chain are not in MS_i, it indicates that the process chain step constraint is unmet, thus assigning a maximum value to m_j. After obtaining the mask values m_j for each step, they are sequentially added to M_i, resulting in the current dynamic mask.

3.2.5 Optimization and usage

The Activist network is represented as a strategy π_θ using parameters θ. An FMS sequencing problem fs is sampled from the dataset FS. The actor network provides the sequencing result τ. Based on the sequencing results of the FMSs, the corresponding costs of fixture adjustment, cutting tool replacement, and non-cutting tool path can be calculated, yielding the total cost C, and the following loss expectation is formulated in conjunction with the cost model C:

$$J(\theta|fs) = \mathbb{E}_{\tau\sim p_\theta(\cdot|fs)} C(\tau|fs) \tag{23}$$

Parameter optimization leverages policy gradient and stochastic gradient descent (SGD). Drawing inspiration from the round update method of REINFORCE [34], the gradient calculation formula for the objective function is given as:

$$\nabla_\theta J(\theta|fs) = \mathbb{E}_{\tau\sim p_\theta(\cdot|fs)}[(C(\tau|fs) - P(fs))\nabla_\theta \log p_\theta(\tau|fs)] \tag{24}$$

where $P(fs)$ represents the optimal cost predicted by the critic network, serving as a baseline for the policy gradient. This ensures that the likelihood of better-performing policies increases, while that of poorly performing policies decreases. During training, a batch of sequencing samples is fed into the actor and critic networks simultaneously. Assuming that there are B sequencing samples in each batch, resulting in B sequencing outcomes and optimal cost predictions from two networks, respectively. The gradient calculation for the actor network is approximately represented by Monte Carlo sampling as follows:

$$\nabla_\theta J(\theta) \approx \frac{1}{B}\sum_{i=1}^{B}(C(\tau_i|fs_i) - P(fs_i))\nabla_\theta \log p_\theta(\tau_i|fs_i) \tag{25}$$

The critic network, denoted as θ_c, is parameterized to predict the expected cost of the current actor network. Serving as a dynamic baseline, it adapts throughout the training process, enhancing the actor network's optimization. The critic network constructs the mean square error based on the difference between the predicted and actual costs, optimizing this error using the SGD method. The loss function for this process is formulated as follows:

$$L(\theta_c) = \frac{1}{B}\sum_{i=1}^{B}\left\|P_{\theta_c}(fs_i) - C(\tau_i|fs_i)\right\|^2 \tag{26}$$

The reinforcement learning process is outlined in the pseudo-code of Algorithm 1. Following T rounds of iterative learning, both the actor network and the critic network achieve convergence.

Algorithm 1 Optimization algorithm for FMS sequencing based on DRL

1: Parameters: FMS dataset ***FS***, Iteration epochs ***T***, Batch size ***B***
2: Initialize the parameters θ of the actor network
3: Initialize the parameters θ_c of the critic network
4: **For** t=1 to ***T*** **do**
5: fs_i ~ Extract ***B*** FMS sequencing problems from the dataset ***FS***
6: $\tau_i \leftarrow$ Input fs_i into the actor network to get the corresponding sequencing result
7: $p_i \leftarrow$ Input fs_i into the critic network to get the corresponding prediction cost
8: $\nabla_\theta \leftarrow$ Gradient calculation for actor network: $\frac{1}{B}\sum_{i=1}^{B}(C(\tau_i \mid fs_i) - P(fs_i))\nabla_\theta \log p_\theta(\tau_i \mid fs_i)$
9: $\nabla_{\theta_c} \leftarrow$ Gradient calculation for critic network: $\frac{1}{B}\sum_{i=1}^{B}\nabla_{\theta_c}\left\|P(fs_i) - C(\tau_i)\right\|^2$
10: $\theta \leftarrow$ ADAM(θ, ∇_θ) Update actor network parameters
11: $\theta_c \leftarrow$ ADAM(θ, ∇_{θ_c}) Update actor network parameters
12: **End for**

After training the DRL framework, the actor network loads the trained parameters to solve the sequencing problem *fs*, because the forward calculation time of the actor network is short. Consequently, the actor network can be used multiple times to get multiple sequencing results, allowing the selection of the least costly sequencing result as the final result, and DRL models have online deep reinforcement learning (ODRL) capabilities. If the process personnel are unsatisfied with the sequencing result τ_θ, iterative learning can continue using the actor network with trained parameters, ultimately achieving a better sequencing outcome. The pseudo-code of Algorithm 2 details the process. To increase sampling randomness, the input order of FMSs in *fs* is disrupted, enhancing the effect of active search. After *N* iterative learnings, the algorithm outputs the sequencing result with the lowest cost found during the search process.

Algorithm 2 Online deep reinforcement learning of actor network

1: Parameters: Sequencing problem *fs*, Sequencing result τ_θ of the actor network, Learning epochs N

2: $C_{\tau_\theta} \leftarrow$ Calculate the cost of sequencing result τ_θ

3: $b \leftarrow$ Take the initial cost C_{τ_θ} as the baseline

4: **For** t=1 to N **do**

5: $\tau_i \leftarrow$ Input *fs* into the actor network *n* times to obtain *n* sequencing results ($1 \le i \le n$)

6: $C_{\tau_i} \leftarrow$ Calculate the cost of all sequencing results τ_i

7: $C_{\tau_j} \leftarrow$ Traverse all costs and find the minimum cost

8: $\tau_j \leftarrow$ Find the sequencing result corresponding to the minimum cost

9: **If** $C_{\tau_j} < C_{\tau_\theta}$ **then**

10: $\tau_\theta \leftarrow \tau_j$

11: $C_{\tau_\theta} \leftarrow C_{\tau_j}$

12: **End if**

13: $\nabla_\theta \leftarrow$ Gradient calculation for actor network: $\frac{1}{B}\sum_{i=1}^{B}(C(\tau_i)-b)\nabla_\theta \log p_\theta(\tau_i \mid fs)$

14: $\theta \leftarrow$ ADAM(θ, ∇_θ) Update actor network parameters

15: $b \leftarrow \alpha \cdot b + (1-\alpha)\cdot \frac{1}{n}\sum_{i=1}^{n} C_{\tau_i}$

16: **End for**

17: Output τ_θ

3.3 Dataset construction

The limitation of the proposed feature operation ranking method based on deep reinforcement learning is that it requires a large number of unlabeled samples for network parameter training. A search of the literature and online resources reveals that no publicly available feature process step sorting dataset exists. To address this issue, this section proposes an automatic construction method for FMS datasets. The method includes the following steps:

Step 1: Construct the grid space. A virtual rectangular part is created, and the part is uniformly discretized into a grid space along the coordinate axes. This grid space can be understood as the bounding box of any three-axis part.

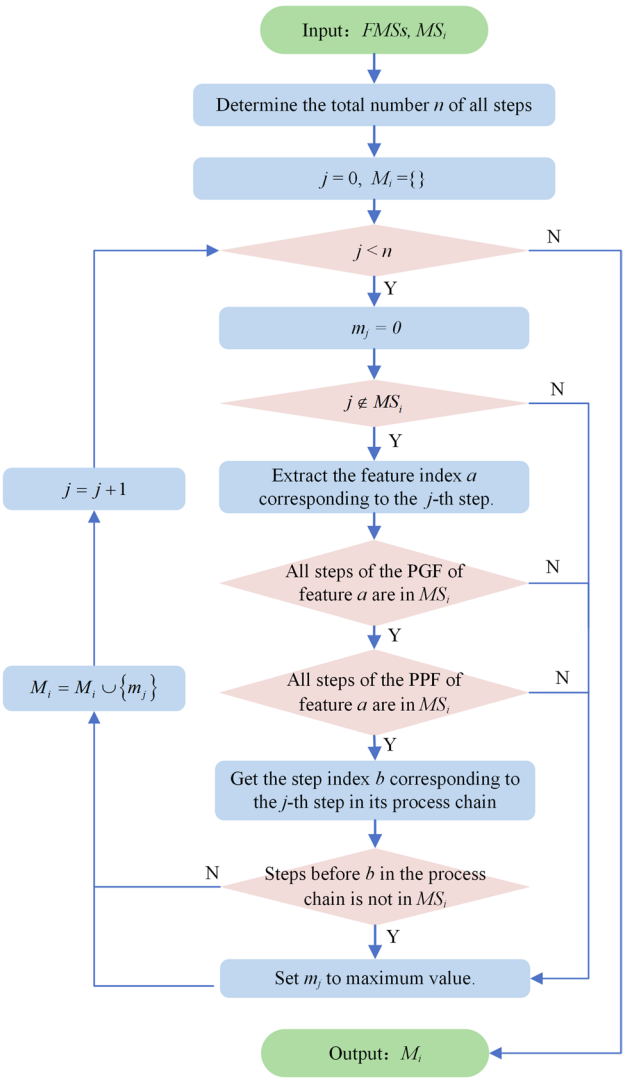


Fig. 5 Construction of a dynamic mask

Step 2: Select a grid within the grid space. A cutting direction is randomly selected, and along that direction, a random grid on the part surface is chosen. If the grid has not previously been assigned a feature, it is selected to generate a feature sample. If the grid already contains a feature with the same cutting direction, the next grid along the cutting direction that does not have a feature is selected for sample generation.

Step 3: Repeat Step 2 to generate the specified number of FMS data. Each feature is randomly assigned coordinate information within the grid, the number of machining steps, and the cutting tools used for each machining step. The geometric dependencies between features located in adjacent grids with the same cutting direction are then established. Finally, the data is organized in the format of Table 1 to obtain the sample data for all features of the current part.

Step 4: Repeating Steps 1–3 to generate the FMS dataset for a specified number of parts.

4 Results and discussion

4.1 Training results

The model is implemented using the PyTorch library. The training process is conducted on a computer equipped with an Intel® Core™ i5-12400F CPU and an NVIDIA GeForce RTX 3060 GPU. Each training batch consists of 256 samples, and the model is trained for a total of 6000 rounds. The training process halts if the average cost of the sequencing results does not exceed the cumulative average cost for 100 consecutive iterations. The training stopped at the 4800th round, taking approximately 60 min. The training results are illustrated in Fig. 6.

As illustrated in Fig. 6a, b, both the actor and critic networks are trained to converge. Figure 6c shows sequencing results violating constraints, with the number of violations always at 0. This demonstrates the dynamic selection mask's effectiveness in ensuring the probability distribution adheres to constraints when selecting the next FMS. Figure 6d–f respectively presents the value of fixture adjustments, cutting tool replacements, and non-cutting tool paths. The convergence curve reveals that the actor network initially learned the fixture-related sequencing experience. For machining features oriented in the same direction, the process aims to complete machining in a single clamping whenever possible. Subsequently, the actor network learns to use the same cutting tool to perform as many FMSs as feasible. Finally, the actor network learns to minimize the non-cutting tool path between FMSs.

The learning process of the actor network aligns with the experience of process personnel during manual sequencing. This alignment occurs because the single-step switching cost is introduced as a local cost in the calculation of the probability distribution, resembling the greedy strategy employed by process personnel during sequencing. However, after obtaining the resembling result, the actor network updates the network parameters based on the global cost, thus avoiding the local optimum induced by the greedy strategy. The sequencing method based on the deep reinforcement learning framework simultaneously introduces local and global costs. This approach not only accelerates the learning process but also prevents the final result from falling into local optima, thereby achieving a more optimal sequencing result. The actor network's learning process mirrors the process personnel's manual sequencing experience by introducing local cost in the probability distribution calculation, similar to the greedy strategy. However, the actor network updates parameters based on the global

cost, avoiding local optima induced by the greedy strategy. The proposed method incorporates both local and global costs, accelerating learning and preventing local optima, thus achieving a more optimal sequencing result.

4.2 Case study

The two parts utilized in the case study are depicted in Fig. 7. Part 1 is a typical model from the repository of the National Institute of Standards and Technology, while Part 2 is a typical model from the commercial software FeatureCAM. The features of Part 1 and Part 2 need to be processed from two directions.

In the developed prototype system, the machining features and related information are automatically recognized using the previously proposed method [35]. Each machining feature is then assigned a TAD, the number of FMSs, and the cutting tool used for each FMS. The shape dependencies between features have also been obtained during the feature recognition process (Fig. 8). The detailed information of Part 1 can be found in Appendix 1, while that of Part 2 is available in Appendix 2. The result of FMS planning must meet the constraints of the preceding feature and process chain steps.

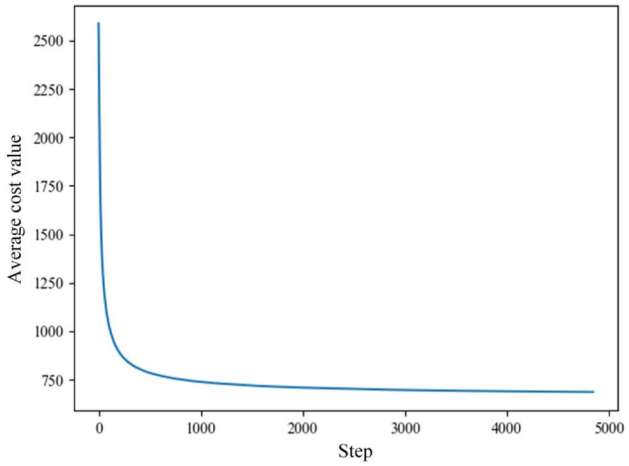
To verify the superiority of the proposed framework, the two parts were used to compare the proposed method with other methods. Specifically, the approximate methods employed the GA algorithm and the edge selection-based genetic algorithm (ESGA) [17]. To enhance the population evolution efficiency of the GA algorithm, a constraint violation cost with a maximum value was introduced into the individual cost. ESGA uses the edge selection strategy to significantly enhance population evolution efficiency and the feasibility of sequencing results. According to reference [17], the population size for both the GA and ESGA algorithms was set to 150. The number of iterations was determined to be 500 based on experiments. For exact methods, the branch-and-bound (BB) and rule-based branch-and-bound (RBB) algorithms [14] were used for comparison. To enhance the search efficiency of these algorithms, a constraint violation stop search mechanism was incorporated. In addition, the greedy method mirrors the manual sequencing strategy of process personnel, selecting the step with the least cost under the condition of not violating the constraint. Thus, the results of the greedy algorithm can serve as a benchmark for manual sequencing. To ensure fairness in comparison, all algorithms were executed on the same CPU.

The sequencing results for Part 1 and Part 2 are presented in Table 2. The GA and ESGA algorithms, being heuristic methods, were run 100 times each, and the average of their feasible solutions was used for comparison. The BB method and the RBB method are exact methods for searching the solution space and only need to be run once. The DRL method uses the trained network model to get the sequencing result. The ODRL method provides the continuing online learning result after the DRL method is run once. The number of online learning is a hyperparameter and is set to 10. The specific FMS sequencing results of the DRL and ODRL methods are available in Appendix 3.

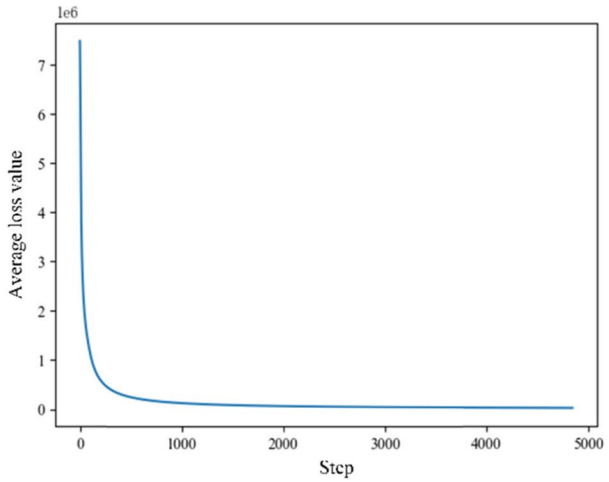
From the results of Part 1, the GA algorithm cannot guarantee that all the sequencing results are feasible. Although the results of the ESGA algorithm are feasible solutions, its optimization results are not ideal. The computation time and the sequencing result of the Greedy method are acceptable. But the optimization effect of the Greedy method is limited. For the 28-step sequencing problem of Part 1, the BB and RBB algorithms were halted after 12 h due to the excessively large solution space. This extended computation time renders these methods impractical for FMS sequencing. In contrast, our DRL method shows better total cost and shorter computation time compared to GA and ESGA. Although the DRL algorithm's computation time is longer than that of the Greedy method, it reduces the non-cutting tool path by 23.99%. Additionally, the DRL algorithm's computation time of 1.19 s is acceptable. If time permits, the ODRL method can continue running and performing online learning to achieve better sequencing results.

The number of FMSs increases from 29 in Part 1 to 58 in Part 2. Sequencing results show that, as the problem scale grows, the computation time for the GA and ESGA rises significantly, whereas the DRL method's computation time remains nearly unchanged. Under these conditions, the computation time of the traditional method is a maximum of 56 s, which initially appears to be within an acceptable range. However, as the number of features in the part increases, the computation time will grow exponentially, ultimately exceeding the acceptable limits for practical applications. The optimization result of the DRL method is the best. Compared to the Greedy method, the non-cutting tool path is reduced by 10.16%. The ODRL method finds better sequencing results. If time permits and the user aims to find better results, the ODRL algorithm will be used.

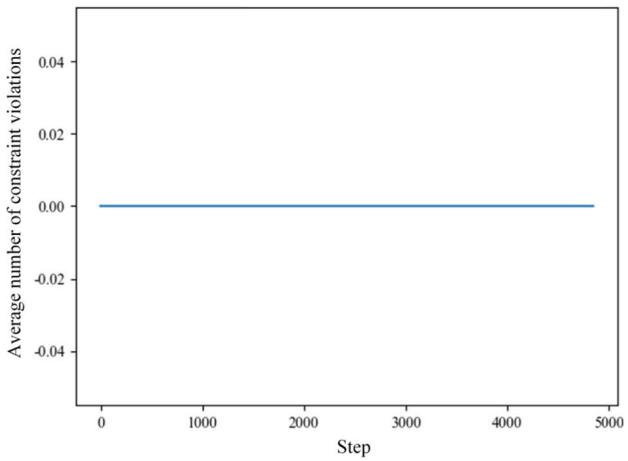
Sequencing results from both parts demonstrate the DRL method's effectiveness in FMS sequencing. As the number of steps increases, the DRL algorithm's computation time remains nearly constant, with both computation time and optimization results acceptable for practical engineering. Furthermore, the ODRL algorithm can achieve better sequencing results with additional computing time.



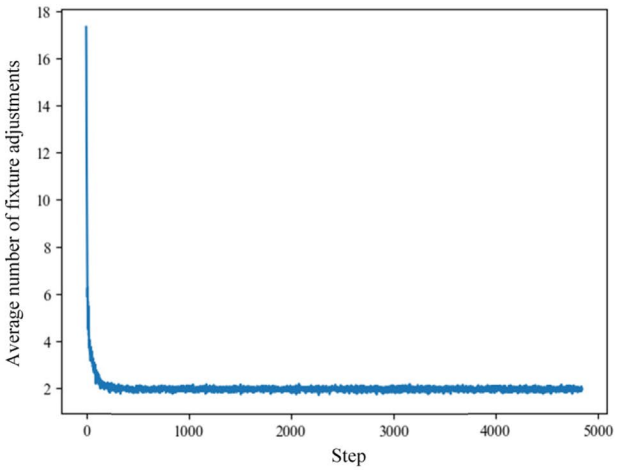
(a) Average cost of actor network sequencing results



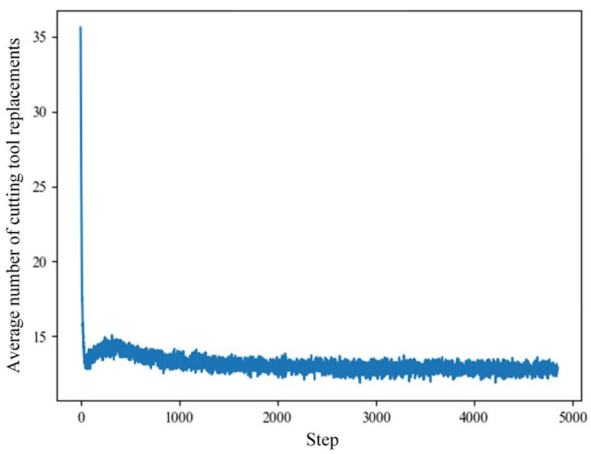
(b) Average loss value of critic network



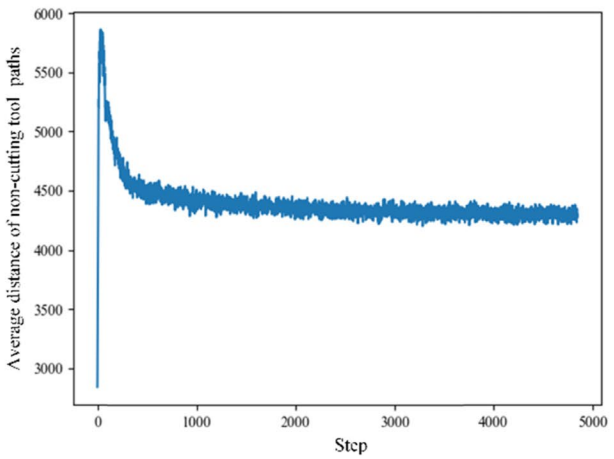
(c) Average number of constraint violations



(d) Average number of fixture adjustments

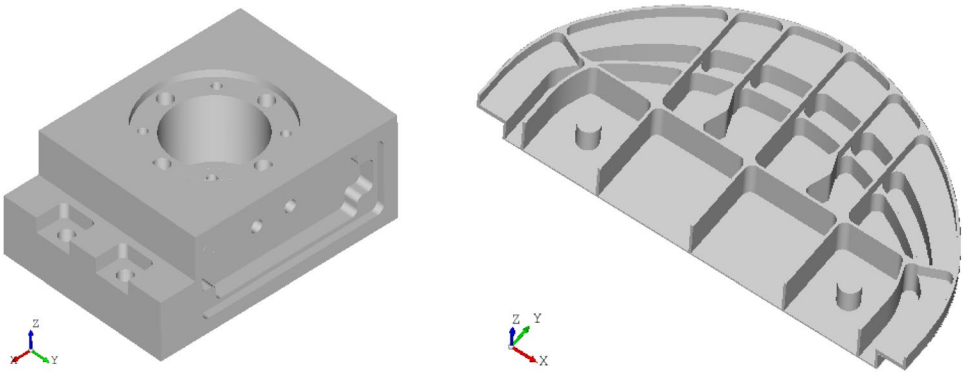


(e) Average number of cutting tool replacements



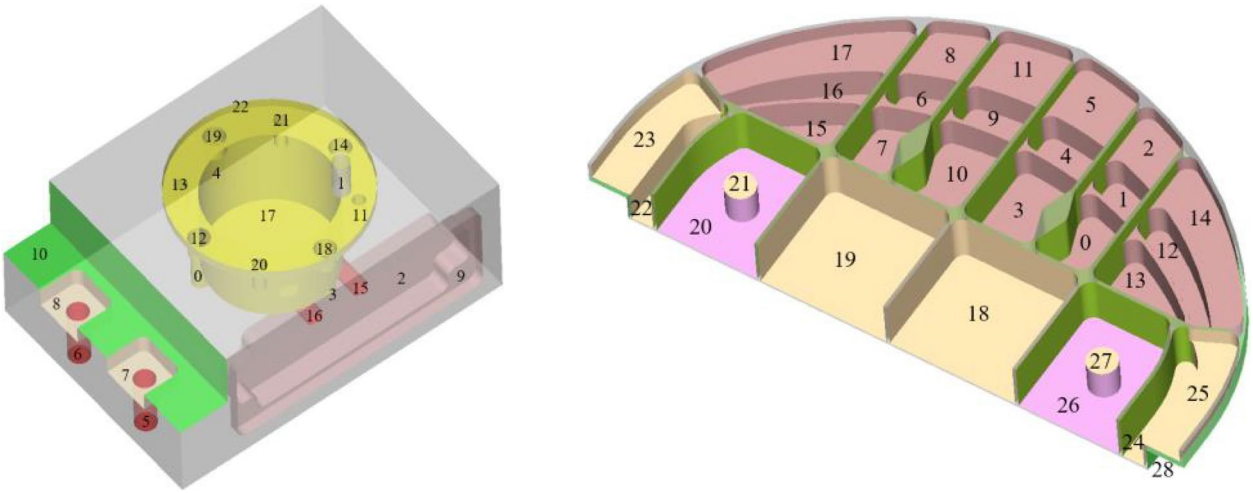
(f) Average distance of non-cutting tool paths

Fig. 6 Training results



(a) Model of Part 1 (b) Model of Part 2

Fig. 7 Part models used in the case study



(a) Features of Part 1 (b) Features of Part 2

Fig. 8 Feature recognition results

Table 2 Comparison of sequencing results using different methods

Part	Result	GA	ESGA	Greedy	BB	RBB	DRL (ours)	ODRL (ours)
1	Feasible probability	79%	100%	100%	-	-	100%	100%
	Time (sec)	17 s	4.94 s	0.002 s	> 12 h	> 12 h	1.19 s	18.52 s
	Total cost	583.64	371.85	321.87	-	-	307.03	305.99
	Fixture Adjustment	3.95	1.97	1	-	-	1	1
	Tool replacement	5.73	5.07	8	-	-	8	8
	Non-cutting tool path (mm)	1480.56	1468.91	1237.43	-	-	940.52	919.83
2	Feasible probability	23%	100%	100%	-	-	100%	100%
	Time (sec)	56 s	11.63 s	0.006 s	> 12 h	> 12 h	4.83 s	384.75 s
	Total cost	706.268	432.142	433.41	-	-	409.70	401.35
	Fixture Adjustment	2.87	1.050	1	-	-	1	1
	Tool replacement	8.13	5.710	5	-	-	5	5
	Non-cutting tool path (mm)	5134.06	4258.84	4668.19	-	-	4193.92	4026.98

5 Conclusions and future works

To address the challenge of optimizing the sequence of FMSs, a knowledge-guided deep reinforcement learning framework was proposed in this paper. The primary work and contributions include the following: (1) Considering the constraints, a cost model based on process knowledge is constructed. (2) A knowledge-guided deep reinforcement learning framework consisting of AR and Pt modules is proposed. By introducing local and global costs, the framework ensures quick convergence and better sequencing results. Additionally, the dynamic selection mask is also introduced to ensure adherence to constraints. (3) Compared to existing methods, the DRL method performs well in both total cost and computation time. As the number of steps increases, the computation time of the DRL algorithm remains relatively stable, and its optimization performance satisfies actual engineering requirements. The ODRL algorithm further refines the sequencing results through additional learning and optimization.

While the proposed framework has effectively addressed the FMS sequencing problem on a single milling machine, further research is necessary to extend this approach to more complex multi-machine manufacturing scenarios. Future work will focus on sequencing FMSs for parts with hybrid process requirements, such as those involving both milling and turning operations on different machine tools. A key objective will be to leverage model information and process knowledge to optimize the coordination between machines, thereby minimizing non-cutting switching time across multiple machines and further enhancing overall machining efficiency.

Appendix 1. Geometric and process information Part 1

The top center coordinates of all features in Part 1 are listed in Table 3. All features are rotated to the $+Z$ direction to calculate the top center coordinates. The relative positions of features in the same TAD remain unchanged, which is convenient for using the non-cutting tool path function.

The process personnel designed the feature process chain for all features, as shown in Table 4. Each FMS is assigned a global step index and cutting tool index. The TAD index is determined according to the feature orientation. The global index is the unique index of the step in all steps. Each step also has a unique local index in its process chain. The local index refers to the position of the step within its feature process chain, as well as its machining order. For example, the process chain of the pocket feature with index 2 contains 2 FMSs. The index of the roughing step executed first is 0, and the index of the finishing step executed later is 1. The geometric dependency between features can be obtained based on the feature recognition result and process reference information. For example, the pocket feature with index 9 is the pre-machining feature of the pocket feature with index 2, and the two have a geometric dependency.

Table 3 Coordinates of the features in Part 1

Feature ID	Type	Preceding feature ID	*X* (mm)	*Y* (mm)	*Z* (mm)
0	Blind hole	12	112.50	60.00	55.05
1	Blind hole	14	37.50	60.00	55.08
2	Pocket	9	77.78	28.28	115.00
3	Blind hole	18	75.00	97.50	55.08
4	Blind hole	19	75.00	22.50	55.08
5	Through hole	7	160.00	82.50	30.12
6	Through hole	8	160.00	37.50	30.12
7	Blind hole	10	160.00	82.5	40.10
8	Blind hole	10	160.00	37.50	40.10
9	Pocket	None	77.78	28.28	120.00
10	Step	None	157.50	60.00	70.48
11	Blind hole	22	48.48	86.52	65.00
12	Blind hole	22	112.50	60.00	65.00
13	Blind hole	22	101.52	33.48	65.00
14	Blind hole	22	37.50	60.00	65.00
15	Through hole	2	69.30	19.80	105.00
16	Through hole	2	86.27	36.77	105.00
17	Blind hole	22	75.00	97.50	65.04
18	Blind hole	22	75.00	97.50	65.04
19	Blind hole	22	75.00	22.50	65.04
20	Blind hole	22	101.52	86.52	65.06
21	Blind hole	22	48.48	33.48	65.06
22	Blind hole	None	75.00	60.00	70.36

Table 4 Process planning information of the features in Part 1

Feature ID	Local ID	Global ID	Description	TAD	Tool ID
0	0	0	Finishing	−*Z*	2
1	0	1	Finishing	−*Z*	2
2	0	2	Roughing	−*Y*	0
	1	3	Finishing	−*Y*	1
3	0	4	Finishing	−*Z*	2
4	0	5	Finishing	−*Z*	2
5	0	6	Finishing	−*Z*	1
6	0	7	Finishing	−*Z*	1
7	0	8	Finishing	−*Z*	1
8	0	9	Finishing	−*Z*	1
9	0	10	Roughing	−*Y*	0
	1	11	Finishing	−*Y*	1
10	0	12	Roughing	−*Z*	0
	1	13	Finishing	−*Z*	1
11	0	14	Finishing	−*Z*	2
12	0	15	Finishing	−*Z*	1
13	0	16	Finishing	−*Z*	2
14	0	17	Finishing	−*Z*	1
15	0	18	Finishing	−*Y*	1
16	0	19	Finishing	−*Y*	1
17	0	20	Roughing	−*Z*	0
	1	21	Bottom finishing	−*Z*	1
	2	22	Wall finishing	−*Z*	1
18	0	23	Finishing	−*Z*	1
19	0	24	Finishing	−*Z*	1
20	0	25	Finishing	−*Z*	2
21	0	26	Finishing	−*Z*	2
22	0	27	Roughing	−*Z*	0
	1	28	Finishing	−*Z*	1

Appendix 2. Geometric and process information Part 2

The top center coordinates of all features in Part 2 are listed in Table 5. All features are rotated to the $+Z$ direction to calculate the top center coordinates.

Similarly, the feature process chain of Part 2 is given as shown in Table 6. The process chain specified for each feature includes all FMSs, and the steps are assigned a local index according to the machining order in the chain. Each feature step is assigned a unique global index, and the TAD and cutting tool indices are specified. For example, the process chain of the pocket feature with an index of 2 contains 2 steps. The index of the roughing step executed first is 0, and the index of the finishing step executed later is 1.

Table 5 Coordinates of the features in Part 2

Feature ID	Type	Preceding feature ID	*X* (mm)	*Y* (mm)	*Z* (mm)
0	Pocket	2	70.00	0.70	−10.00
1	Pocket	2	70.00	41.96	−11.15
2	Pocket	None	70.00	32.46	0.92
3	Pocket	5	25.00	4.44	−11.02
4	Pocket	5	25.00	51.37	−11.03
5	Pocket	None	25.00	38.30	1.01
6	Pocket	8	−70.00	41.96	−11.15
7	Pocket	8	−70.00	0.70	−10.00
8	Pocket	None	−70.00	32.46	1.80
9	Pocket	11	−25.00	51.37	−11.05
10	Pocket	11	−25.00	4.43	−11.02
11	Pocket	None	−25.00	38.30	1.01
12	Pocket	14	131.51	7.55	−11.27
13	Pocket	14	122.04	−6.71	−10.75
14	Pocket	None	139.22	23.17	2.01
15	Pocket	17	−122.05	−6.71	−10.75
16	Pocket	17	−131.51	7.55	−11.27
17	Pocket	None	−139.22	23.17	2.01
18	Blind slot	None	45.00	−69.64	2.01
19	Blind slot	None	−45.00	−69.64	2.01
20	Interacting feature	21	−126.25	−69.64	−10.3
21	Blind slot	None	−126.25	−69.64	1.97
22	Blind slot	23	−167.41	−69.64	−8.28
23	Blind slot	None	−180.54	−64.67	2.01
24	Blind slot	25	167.41	−69.64	−8.28
25	Blind slot	None	180.054	−64.67	2.01
26	Interacting feature	27	126.25	−69.64	−10.03
27	Blind slot	None	126.25	−69.64	1.97
28	Step	None	−44.22	44.2215	37.00

Appendix 3. FMS sequencing results

The FMS sequencing results of Part 1 and Part 2 processed by DRL and ODRL methods are presented in Table 7. The parameters within parentheses represent feature identifiers: the first value indicates the feature ID, and the second value corresponds to the local FMS ID within the feature.

Table 6 Process planning information of the features in Part 2

Feature ID	Local FMS ID	Global FMS ID	Description	TAD	Tool ID
0	0	0	Roughing	$+Z$	0
	1	1	Finishing	$+Z$	1
1	0	2	Roughing	$+Z$	0
	1	3	Finishing	$+Z$	1
2	0	4	Roughing	$+Z$	0
	1	5	Finishing	$+Z$	1
3	0	6	Roughing	$+Z$	0
	1	7	Finishing	$+Z$	1
4	0	8	Roughing	$+Z$	0
	1	9	Finishing	$+Z$	1
5	0	10	Roughing	$+Z$	0
	1	11	Finishing	$+Z$	1
6	0	12	Roughing	$+Z$	0
	1	13	Finishing	$+Z$	1
7	0	14	Roughing	$+Z$	0
	1	15	Finishing	$+Z$	1
8	0	16	Roughing	$+Z$	0
	1	17	Finishing	$+Z$	1
9	0	18	Roughing	$+Z$	0
	1	19	Finishing	$+Z$	1
10	0	20	Roughing	$+Z$	0
	1	21	Finishing	$+Z$	1
11	0	22	Roughing	$+Z$	0
	1	23	Finishing	$+Z$	1
12	0	24	Roughing	$+Z$	0
	1	25	Finishing	$+Z$	1
13	0	26	Roughing	$+Z$	0
	1	27	Finishing	$+Z$	1
14	0	28	Roughing	$+Z$	0
	1	29	Finishing	$+Z$	1
15	0	30	Roughing	$+Z$	0
	1	31	Finishing	$+Z$	1
16	0	32	Roughing	$+Z$	0
	1	33	Finishing	$+Z$	1
17	0	34	Roughing	$+Z$	0
	1	35	Finishing	$+Z$	1
18	0	36	Roughing	$+Z$	0
	1	37	Finishing	$+Z$	1
19	0	38	Roughing	$+Z$	0
	1	39	Finishing	$+Z$	1
20	0	40	Roughing	$+Z$	0
	1	41	Finishing	$+Z$	1
21	0	42	Roughing	$+Z$	0
	1	43	Finishing	$+Z$	1
22	0	44	Roughing	$+Z$	0
	1	45	Finishing	$+Z$	1
23	0	46	Roughing	$+Z$	0
	1	47	Finishing	$+Z$	1
24	0	48	Roughing	$+Z$	0
	1	49	Finishing	$+Z$	1

Table 6 (continued)

Feature ID	Local FMS ID	Global FMS ID	Description	TAD	Tool ID
25	0	50	Roughing	$+Z$	0
	1	51	Finishing	$+Z$	1
26	0	52	Roughing	$+Z$	0
	1	53	Finishing	$+Z$	1
27	0	54	Roughing	$+Z$	0
	1	55	Finishing	$+Z$	1
28	0	56	Roughing	$-Z$	0
	1	57	Finishing	$-Z$	1

Table 7 FMS sequencing results of two parts

Part	DRL sequencing results	ODRL sequencing results
1	(10,0) (22,0) (10,1) (7,0) (5,0) (8,0) (6,0) (22,1) (12,0) (18,0) (14,0) (19,0) (0,0) (20,0) (13,0) (21,0) (4,0) (1,0) (11,0) (3,0) (17,0) (17,1) (17,2) (9,0) (9,1) (2,0) (2,1) (16,0) (15,0)	(10,0) (22,0) (10,1) (7,0) (5,0) (8,0) (6,0) (22,1) (12,0) (18,0) (14,0) (19,0) (13,0) (0,0) (20,0) (11,0) (3,0) (4,0) (21,0) (1,0) (17,0) (17,1) (17,2) (9,0) (9,1) (2,0) (2,1) (16,0) (15,0)
2	(25,0) (27,0) (18,0) (14,0) (2,0) (5,0) (11,0) (8,0) (17,0) (21,0) (23,0) (19,0) (18,1) (27,1) (25,1) (14,1) (2,1) (5,1) (11,1) (8,1) (17,1) (21,1) (23,1) (19,1) (12,0) (13,0) (26,0) (24,0) (0,0) (1,0) (4,0) (3,0) (9,0) (10,0) (6,0) (7,0) (15,0) (16,0) (20,0) (22,0) (24,1) (26,1) (13,1) (12,1) (0,1) (1,1) (3,1) (4,1) (9,1) (6,1) (10,1) (7,1) (15,1) (16,1) (20,1) (22,1) (28,0) (28,1)	(27,0) (25,0) (14,0) (2,0) (5,0) (18,0) (19,0) (21,0) (23,0) (17,0) (8,0) (11,0) (27,1) (25,1) (14,1) (2,1) (5,1) (11,1) (8,1) (17,1) (23,1) (21,1) (19,1) (18,1)(24,0) (26,0) (13,0) (12,0) (0,0) (1,0) (3,0) (4,0) (9,0) (10,0) (7,0) (6,0) (16,0) (15,0) (20,0) (22,0)(24,1) (26,1) (13,1) (12,1) (0,1) (1,1) (4,1) (3,1) (10,1) (9,1) (6,1) (7,1) (16,1) (15,1) (20,1) (22,1) (28,0) (28,1)

Acknowledgements The authors gratefully acknowledge the financial support of the National Natural Science Foundation of China (52205568) and the Natural Science Research Start-up Foundation of Recruiting Talents of Nanjing University of Posts and Telecommunications (No. NY224047).

Funding The authors received financial support from the National Natural Science Foundation of China (52205568) and the Natural Science Research Start-up Foundation of Recruiting Talents of Nanjing University of Posts and Telecommunications (No. NY224047).

Declarations

Competing interests The authors declare no competing interests.

References

1. Stavropoulos P, Tzimanis K, Souflas T, Bikas H (2022) Knowledge-based manufacturability assessment for optimization of additive manufacturing processes based on automated feature recognition from CAD models. Int J Adv Manuf Technol 122:993–1007. https://doi.org/10.1007/s00170-022-09948-w
2. Zhang H, Wang W, Zhang S, Wang Z, Zhang Y, Zhou J, Huang B (2024) Point cloud self-supervised learning for machining feature recognition. J Manuf Syst 77:78–95
3. Ji W, Wang L, Haghighi A, Givehchi M, Liu X (2018) An enriched machining feature based approach to cutting tool selection. Int J Comput Integr Manuf 31:1–10. https://doi.org/10.1080/0951192X.2017.1356472
4. Machado AR, da Silva LRR, Pimenov DY, de Souza FCR, Kuntoğlu M, de Paiva RL (2024) Comprehensive review of advanced methods for improving the parameters of machining steels. J Manuf Process 125:111–142. https://doi.org/10.1016/j.jmapro.2024.07.044
5. Ji W, Wang L, Haghighi A, Givehchi M, Liu X (2016) A reachability based approach for machining feature sequencing. J Manuf Syst 40:96–104. https://doi.org/10.1016/j.jmsy.2016.06.006
6. Wang P, Liu W, You Y, Qian S (2024) An automatic generation approach of process model based on feature knowledge and geometric modeling. Adv Eng Inform 62:102881. https://doi.org/10.1016/j.aei.2024.102881
7. Liang J, Zhang S, Huang B, Zhang Y, Huang R (2022) Nc process analysis–based intersecting machining feature recognition and reuse approach. Int J Adv Manuf Technol 123:2393–2413. https://doi.org/10.1007/s00170-022-10281-5
8. Chen L, Hu P, Luo M, Tang K (2018) Optimal interface surface determination for multi-axis freeform surface machining with both roughing and finishing. Chin J Aeronaut 31:370–384. https://doi.org/10.1016/j.cja.2017.07.004
9. Deja M, Siemiatkowski MS (2018) Machining process sequencing and machine assignment in generative feature-based CAPP for mill-turn parts. J Manuf Syst 48:49–62. https://doi.org/10.1016/j.jmsy.2018.06.001
10. Tian X, He Y, Li Y, Wang Y, Tao F, Wang Y (2024) Joint optimization of feature sequences and toolpath strategies in multi-feature workpiece machining for minimizing energy consumption and processing time. J Manuf Syst 74:869–886. https://doi.org/10.1016/j.jmsy.2024.05.007
11. Shaw K, Shankar R, Yadav SS, Thakur LS (2012) Supplier selection using fuzzy AHP and fuzzy multi-objective linear programming for developing low carbon supply chain. Expert Syst Appl 39:8182–8192. https://doi.org/10.1016/j.eswa.2012.01.149

12. Mahmoudimehr J, Sebghati P (2019) A novel multi-objective dynamic programming optimization method: performance management of a solar thermal power plant as a case study. Energy 168:796–814. https://doi.org/10.1016/j.energy.2018.11.079
13. Morrison DR, Jacobson SH, Sauppe JJ, Sewell EC (2016) Branch-and-bound algorithms: a survey of recent advances in searching, branching, and pruning. Discret Optim 19:79–102. https://doi.org/10.1016/j.disopt.2016.01.005
14. Wang W, Li Y, Huang L (2018) Rule and branch-and-bound algorithm based sequencing of machining features for process planning of complex parts. J Intell Manuf 29:1329–1336. https://doi.org/10.1007/s10845-015-1181-y
15. Hao X, Li Y, Zhao Z, Liu C (2019) Dynamic machining process planning incorporating in-process workpiece deformation data for large-size aircraft structural parts. Int J Comput Integr Manuf 32:136–147. https://doi.org/10.1080/0951192X.2018.1529431
16. Wang C, Zhang S, Ma T, Xiao Y, Chen MZ, Wang L (2025) Swarm intelligence: a survey of model classification and applications. Chin J Aeronaut 38:102982. https://doi.org/10.1016/j.cja.2024.03.019
17. Su Y, Chu X, Chen D, Sun X (2018) A genetic algorithm for operation sequencing in CAPP using edge selection based encoding strategy. J Intell Manuf 29:313–332. https://doi.org/10.1007/s10845-015-1109-6
18. Katoch S, Chauhan SS, Kumar V (2021) A review on genetic algorithm: past, present, and future. Multimed Tools Appl 80:8091–8126. https://doi.org/10.1007/s11042-020-10139-6
19. Wen X, Qian Y, Lian X, Zhang Y, Wang H, Li H (2023) Improved genetic algorithm based on multi-layer encoding approach for integrated process planning and scheduling problem. Robot Comput Integr Manuf 84:102593. https://doi.org/10.1016/j.rcim.2023.102593
20. Neumann A, Hajji A, Rekik M, Pellerin R (2024) Genetic algorithms for planning and scheduling engineer-to-order production: a systematic review. Int J Prod Res 62:2888–2917. https://doi.org/10.1080/00207543.2023.2237122
21. Hu Q, Qiao L, Peng G (2017) An ant colony approach to operation sequencing optimization in process planning. Proc Inst Mech Eng Part B J Eng Manuf 231:470–489. https://doi.org/10.1177/0954405415616786
22. Xiao Q, Li C, Tang Y, Pan J, Yu J, Chen X (2019) Multi-component energy modeling and optimization for sustainable dry gear hobbing. Energy 187:115911. https://doi.org/10.1016/j.energy.2019.115911
23. Zhang W, Zheng Y, Ahmad R (2023) An energy-efficient multi-objective integrated process planning and scheduling for a flexible job-shop-type remanufacturing system. Adv Eng Inform 56:102010. https://doi.org/10.1016/j.aei.2023.102010
24. Liu Q, Wang C, Li X, Gao L (2023) An improved genetic algorithm with modified critical path-based searching for integrated process planning and scheduling problem considering automated guided vehicle transportation task. J Manuf Syst 70:127–136. https://doi.org/10.1016/j.jmsy.2023.07.004
25. Xu C, Zhang S, Liang J, Rong B, Hou J (2024) Nc process information mining based optimization method of roughing tool sequence selection for pocket features. Adv Eng Inform 61:102501. https://doi.org/10.1016/j.aei.2024.102501
26. Zhang Y, Zhang S, Huang R, Huang B, Liang J, Zhang H, Wang Z (2022) Combining deep learning with knowledge graph for macro process planning. Comput Ind 140:103668. https://doi.org/10.1016/j.compind.2022.103668
27. Han Z, Huang R, Huang B, Jiang J, Li X (2023) Data-driven and knowledge-guided approach for NC machining process planning. Comput Des 162:103562. https://doi.org/10.1016/j.cad.2023.103562
28. Wang Z, Zhang S, Zhang H, Zhang Y, Liang J, Huang R, Huang B (2024) Machining feature process route planning based on a graph convolutional neural network. Adv Eng Inform 59:102249. https://doi.org/10.1016/j.aei.2023.102249
29. Zhang H, Wang W, Zhang S, Zhang Y, Zhou J, Wang Z, Huang B, Huang R (2024) A novel method based on deep reinforcement learning for machining process route planning. Robot Comput Integr Manuf 86:102688. https://doi.org/10.1016/j.rcim.2023.102688
30. Zhang H, Wang W, Wang Y, Zhang Y, Zhou J, Huang B, Zhang S (2025) Employing deep reinforcement learning for machining process planning: an improved framework. J Manuf Syst 78:370–393. https://doi.org/10.1016/j.jmsy.2024.12.010
31. Xiao B, Zhao Z, Xu B, Li Y, Zhang W, Huan H, Su H (2025) A novel method for intelligent reasoning of machining step sequences based on deep reinforcement learning. J Manuf Syst 80:626–642. https://doi.org/10.1016/j.jmsy.2025.04.005
32. Li WD, Ong SK, Nee AYC (2002) Hybrid genetic algorithm and simulated annealing approach for the optimization of process plans for prismatic parts. Int J Prod Res 40:1899–1922. https://doi.org/10.1080/00207540110119991
33. Vinyals O, Fortunato M, Jaitly N (2015) Pointer networks. Adv Neural Inf Process Syst 28. https://doi.org/10.48550/arXiv.1506.03134
34. Williams RJ (1992) Simple statistical gradient-following algorithms for connectionist reinforcement learning. Mach Learn 8:229–256. https://doi.org/10.1007/BF00992696
35. Wang P, Liu W, You Y (2023) A hybrid framework for manufacturing feature recognition from CAD models of 3-axis milling parts. Adv Eng Inform. https://doi.org/10.1016/j.aei.2023.102073

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.